



UNIVERSITAT
Carlemany

Formación europea
para desafíos reales

De:

 Planeta Formación y Universidades

Bloque 2- Desarrollo de APIs REST con ASP.NET Core

Aplicaciones basadas en tecnologías web

Bloque 2 — Desarrollo de APIs REST con ASP.NET Core

- APIs y arquitectura backend
- Recursos y diseño REST
- Métodos HTTP y semántica
- Diseño de endpoints
- ASP.NET Core Web API: visión general
- Controladores y acciones
- Modelos, DTOs y contratos
- Códigos de estado HTTP y manejo de errores
- Arquitectura interna de la API

1. APIs y arquitectura backend

- Qué es una API y qué papel juega en la arquitectura backend
- APIs como frontera entre sistemas y capas
- REST como estilo arquitectónico aplicado a APIs

Qué es una API y qué papel juega en la arquitectura backend

Una **API (Application Programming Interface)** es un **contrato técnico** que define cómo un sistema puede ser utilizado por otros sistemas.

En arquitecturas backend modernas, una API:

- expone funcionalidades de forma controlada
- define **qué se puede hacer y cómo**
- actúa como frontera entre sistemas

Una API:

-  no es una base de datos
-  no es una interfaz gráfica
-  es un **punto de acceso estructurado**

Desde el backend:

- encapsula la lógica de negocio
- protege los datos internos
- permite evolución independiente del cliente

 *Una API bien diseñada es más importante que la tecnología concreta que la implementa.*

APIs como frontera entre sistemas y capas

En una arquitectura profesional, una API funciona como:

- frontera entre **clientes y backend**
- frontera entre **capas internas**
- frontera entre **sistemas distintos**

Esto implica:

- los clientes **no acceden directamente** a la base de datos
- la API controla el acceso y la validación
- los cambios internos no rompen a los consumidores

Desde el punto de vista arquitectónico:

- la API define **qué se expone**
- no **cómo se implementa internamente**

 *La API desacopla quién consume de cómo se implementa.*

REST como estilo arquitectónico aplicado a APIs

REST (Representational State Transfer) no es una tecnología ni un framework. Es un **estilo arquitectónico** para diseñar APIs.

REST propone:

- trabajar con **recursos**
- usar **HTTP** de forma semántica
- mantener el sistema **stateless**

Características clave:

- URLs representan recursos
- métodos HTTP expresan operaciones
- respuestas contienen representaciones del estado

Importante:

- REST no obliga a usar JSON (aunque es común)
- REST no define implementación interna

 *REST es una forma de pensar la API, no una librería.*

2. Recursos y diseño REST

- Qué es un recurso en una API REST
- Identificación de recursos mediante URLs
- Jerarquía de recursos y relaciones en endpoints
- Acciones vs recursos: errores comunes de diseño

Qué es un recurso en una API REST

En REST, un **recurso** es cualquier **entidad o concepto del dominio** que:

- tiene identidad propia
- puede ser referenciado
- puede ser representado

Un recurso:

- no es una acción
- no es un método
- es una “cosa” estable del sistema

Ejemplos conceptuales:

- colecciones de entidades
- elementos individuales
- relaciones con identidad propia

Cada recurso:

- tiene una URL única
- puede tener distintas representaciones

 *Diseñar una API empieza por identificar bien los recursos.*

Identificación de recursos mediante URLs

Las **URLs** sirven para identificar recursos de forma única.

En REST:

- una URL representa **qué es el recurso**
- no **qué operación se realiza**

Buenas prácticas:

- usar sustantivos, no verbos
- evitar acciones en la URL
- mantener estructura predecible

Ejemplo:

/resources

/resources/{id}

Una buena URL:

- es estable en el tiempo
- expresa jerarquía cuando existe
- se puede leer y entender

 *Si la URL parece una acción, el diseño suele ser incorrecto.*

Jerarquía de recursos y relaciones en endpoints

Los recursos pueden tener **relaciones jerárquicas**.

Ejemplo:

/parents

/parents/{id}/children

Esta jerarquía indica:

- relación clara entre recursos
- contexto de acceso
- alcance del recurso hijo

No siempre es necesario:

- solo cuando la relación aporta significado
- no duplicar rutas innecesarias

 *La jerarquía en URLs debe reflejar relaciones reales del dominio.*

Acciones vs recursos: errores comunes de diseño

Un error frecuente es diseñar APIs **orientadas a acciones**.

Ejemplos incorrectos:

/getItem
/createItem
/deleteItem

Por qué es un problema:

- mezcla URL y operación
- ignora semántica HTTP
- dificulta evolución de la API

Diseño correcto:

- URL identifica recurso
- método HTTP define la acción

 *En REST, la acción va en el método HTTP, no en la URL.*

3. Métodos HTTP y semántica

- Semántica de GET, POST, PUT, DELETE y PATCH
- Diferencia entre PUT y PATCH en APIs REST
- Idempotencia y seguridad de los métodos HTTP



Semántica de GET, POST, PUT, DELETE y PATCH

Los **métodos HTTP** definen **la intención de la operación**, no la URL.

Semántica básica:

- **GET** → obtener una representación de un recurso
- **POST** → crear un nuevo recurso o ejecutar una operación no idempotente
- **PUT** → reemplazar completamente un recurso existente
- **DELETE** → eliminar un recurso
- **PATCH** → modificar parcialmente un recurso

Importante:

- el método define el comportamiento esperado
- no es solo “qué hace el servidor”, sino “qué espera el cliente”



Elegir mal el método HTTP rompe el contrato de la API.



Diferencia entre PUT y PATCH en APIs REST

Aunque ambos modifican recursos, **PUT y PATCH no son lo mismo.**

PUT:

- envía la representación completa
- reemplaza el estado del recurso
- es idempotente

PATCH:

- envía solo los cambios
- modifica parcialmente el recurso
- no siempre es idempotente

Decisión de diseño:

- usar PUT cuando el cliente controla todo el estado
- usar PATCH cuando el cambio es parcial o incremental



PUT reemplaza, PATCH ajusta.



Idempotencia y seguridad de los métodos HTTP

Dos conceptos clave en HTTP:

Idempotencia

- ejecutar la misma petición varias veces
- produce siempre el mismo resultado

Ejemplos:

- GET, PUT, DELETE → idempotentes
- POST → no idempotente

Seguridad

- una operación es segura si no modifica estado
- GET es seguro
- POST, PUT, DELETE no lo son



Estas propiedades afectan a caché, reintentos y arquitectura.

4. Diseño de endpoints

- Qué es un endpoint y qué responsabilidad debe tener
- Diseño de endpoints CRUD orientados a recursos
- Endpoints no CRUD: consultas y operaciones de dominio
- Cuándo crear un endpoint nuevo y cuándo no

Qué es un endpoint y qué responsabilidad debe tener

Un **endpoint** es la **combinación concreta** de:

- una URL
- un método HTTP

Un endpoint debe:

- representar **una única responsabilidad clara**
- operar sobre **un recurso o conjunto de recursos**
- devolver una respuesta bien definida

Un endpoint **no debe**:

- contener lógica compleja de negocio
- coordinar demasiadas operaciones
- asumir responsabilidades de persistencia

 *Un endpoint orquesta, no implementa la lógica.*

Diseño de endpoints CRUD orientados a recursos

El diseño CRUD en REST se basa en:

- un recurso principal
- métodos HTTP bien definidos

Estructura conceptual típica:

GET /resources

GET /resources/{id}

POST /resources

PUT /resources/{id}

DELETE /resources/{id}

Claves de buen diseño:

- consistencia entre recursos
- mismo patrón para todos
- comportamiento predecible

 *Un CRUD bien diseñado reduce la curva de aprendizaje de la API.*

Endpoints no CRUD: consultas y operaciones de dominio

No todas las operaciones encajan en CRUD puro.

Ejemplos:

- consultas agregadas
- cálculos de dominio
- operaciones que no modifican un recurso concreto

En estos casos:

- el endpoint sigue representando un **recurso lógico**
- no se convierten acciones en verbos

Ejemplo:

GET /resources/summary

GET /resources/statistics

 *No todo es CRUD, pero todo sigue siendo un recurso.*



Cuándo crear un endpoint nuevo y cuándo no

Crear endpoints sin criterio provoca:

- APIs infladas
- solapamiento de responsabilidades
- contratos difíciles de mantener

Antes de crear un endpoint:

- ¿representa un recurso distinto?
- ¿responde a una pregunta diferente?
- ¿tiene un contrato claro?

Buenas prácticas:

- reutilizar patrones existentes
- evitar endpoints “comodín”
- preferir claridad frente a cantidad



Menos endpoints bien diseñados superan a muchos mal definidos.

5. ASP.NET Core Web API: visión general

- Qué es ASP.NET Core y qué problemas resuelve
- Estructura de un proyecto ASP.NET Core Web API
- El pipeline de ASP.NET Core (request → response)
- De una petición HTTP real a un Controller en ASP.NET Core
- Qué datos entran realmente a un endpoint (.NET binding real)
- Un endpoint bien hecho vs uno mal hecho (comparación real)
- Flujo real dentro de una API ASP.NET Core

Qué es ASP.NET Core y qué problemas resuelve

ASP.NET Core es un framework para construir aplicaciones backend modernas sobre .NET.

Problemas que resuelve:

- creación de APIs HTTP de forma estructurada
- gestión del ciclo request–response
- integración de configuración, logging y seguridad
- soporte multiplataforma

Características clave:

- alto rendimiento
- modularidad
- fuerte orientación a arquitectura por capas

ASP.NET Core **no impone**:

- cómo diseñar tu dominio
- cómo estructurar tu lógica interna

 *El framework proporciona la infraestructura, no el diseño.*

Estructura de un proyecto ASP.NET Core Web API

Un proyecto típico de Web API contiene:

- punto de entrada de la aplicación
- controllers para exponer endpoints
- configuración de servicios
- configuración del pipeline HTTP

Estructura conceptual:

- capa de presentación (API)
- capas internas (aplicación, dominio, infraestructura)

Buenas prácticas:

- no concentrar lógica en los controllers
- separar responsabilidades desde el inicio
- pensar en crecimiento futuro del proyecto

 *La estructura del proyecto refleja la arquitectura del sistema.*

⚙ El pipeline de ASP.NET Core (request → response)

Cada petición HTTP sigue un **pipeline de procesamiento**:

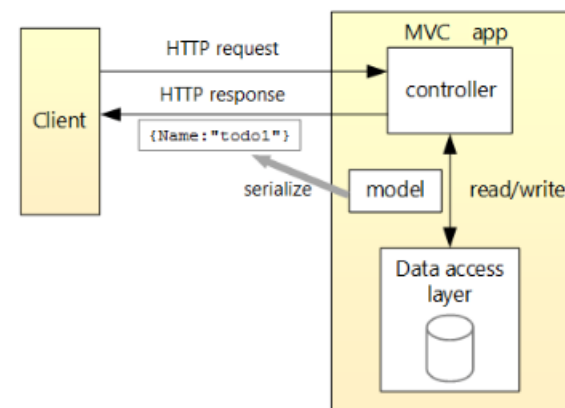
1. la petición entra en el servidor
2. pasa por una cadena de componentes
3. se enruta a un endpoint
4. se genera una respuesta

Este pipeline:

- es configurable
- es ordenado
- permite interceptar peticiones y respuestas

Conceptualmente:

- no todo pasa directamente al controller
- existen pasos previos y posteriores



 Entender el pipeline ayuda a comprender cómo se ejecuta una API.

De una petición HTTP real a un Controller en ASP.NET Core

Caso práctico

Llega esta petición HTTP al backend:

GET /assets/42

Qué ocurre en ASP.NET Core:

- El servidor recibe la petición HTTP.
- El **routing** busca un controller cuya ruta encaje con /assets/{id}.
- Se ejecuta la acción correspondiente pasando id = 42.

Ejemplo conceptual de acción:

```
[HttpGet("{id}")]  
public IActionResult GetById(int id)  
{  
    // lógica delegada  
}
```

Salida esperada:

- 200 OK + JSON si existe
- 404 Not Found si no existe

 *El controller es el punto exacto donde HTTP se convierte en código C#.*

Qué datos entran realmente a un endpoint (.NET binding real)

En ASP.NET Core, los datos pueden llegar desde **tres sitios distintos**:

1 Ruta

bash

```
GET /assets/42
```

csharp

```
public IActionResult Get(int id)
```

2 Query string

bash

```
GET /assets?category=industrial
```

csharp

```
public IActionResult Get(string category)
```

3 Body (JSON)

bash

```
POST /assets
{
  "latitude": 40.4,
  "longitude": -3.7,
  "baseValue": 100000
}
```

csharp

```
public IActionResult Create(CreateAssetDto dto)
```

 *ASP.NET Core convierte automáticamente HTTP en parámetros C#.*

Un endpoint bien hecho vs uno mal hecho (comparación real)

Endpoint mal diseñado

```
csharp

[HttpGet("getAssetAndCalculate")]
public IActionResult GetAsset(int id)
{
    // accede a BD
    // calcula métricas
    // aplica reglas
}
```

Endpoint bien diseñado

```
csharp

[HttpGet("{id}")]
public IActionResult GetById(int id)
{
    var asset = assetService.GetById(id);
    if (asset == null) return NotFound();
    return Ok(asset);
}
```

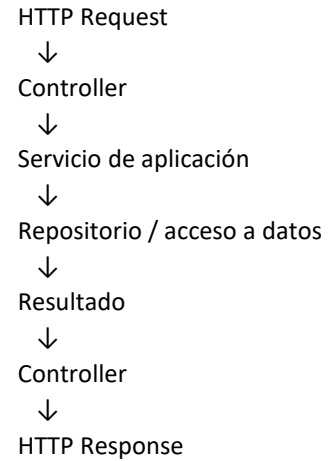
Problemas:

- mezcla responsabilidades
- difícil de mantener
- imposible de reutilizar

 *El endpoint coordina, la lógica vive fuera.*

Flujo real dentro de una API ASP.NET Core

Flujo típico de una petición real:



Ejemplo de salida:

```
200 OK
{
  "id": 42,
  "baseValue": 100000
}
```

Decisión clave:

- el controller **nunca** sabe cómo se guardan los datos
- solo sabe **qué hacer con el resultado**

 *Este flujo se repite en todos los endpoints bien diseñados.*

6. Controladores y acciones

- Qué es un Controller en ASP.NET Core
- Acciones: cómo un método se convierte en un endpoint
- IActionResult: decidir la respuesta HTTP
- Crear recursos: POST con input y output claros
- Actualizar recursos: PUT vs PATCH (caso real)
- Un endpoint bien hecho vs uno mal hecho (comparación real)
- Eliminar recursos: DELETE bien diseñado
- Errores comunes en controllers (muy reales)

Qué es un Controller en ASP.NET Core (en la práctica)

Un **Controller** es la clase que **recibe peticiones HTTP** y **devuelve respuestas HTTP**.

Ejemplo:

```
csharp
[ApiController]
[Route("assets")]
public class AssetsController : ControllerBase
{
}
```

Qué significa esto:

- [ApiController] → activa validaciones y comportamientos REST
- [Route("assets")] → prefijo común para todos los endpoints
- hereda de ControllerBase → solo API, sin vistas

 *Un controller representa un recurso principal de la API.*



Acciones: cómo un método se convierte en un endpoint

Cada **método público** del controller puede ser un endpoint.

Ejemplo:

```
csharp

[HttpGet("{id}")]
public IActionResult GetById(int id)
{
    return Ok();
}
```

Esto genera:

GET /assets/5

Claves importantes:

- el atributo (HttpGet) define el método HTTP
- la ruta ({id}) se enlaza con el parámetro
- el nombre del método **no importa** para el cliente



El endpoint lo define el atributo, no el nombre del método.

IActionResult: decidir la respuesta HTTP

IActionResult permite devolver **respuestas HTTP distintas** según el caso.

Ejemplo:

```
csharp

[HttpGet("{id}")]
public IActionResult GetById(int id)
{
    var asset = service.GetById(id);

    if (asset == null)
        return NotFound();

    return Ok(asset);
}
```

Resultados posibles:

- 200 OK + JSON → recurso encontrado
- 404 Not Found → recurso inexistente

 *La API comunica estado mediante códigos HTTP, no solo datos.*



Crear recursos: POST con input y output claros

Ejemplo de creación:

```
POST /assets
{
  "latitude": 40.4,
  "longitude": -3.7,
  "baseValue": 100000
}
```

Acción correspondiente:

```
csharp

[HttpPost]
public IActionResult Create(CreateAssetDto dto)
{
    var id = service.Create(dto);
    return CreatedAtAction(nameof(GetById), new { id }, null);
}
```

Respuesta:

```
201 Created
Location: /assets/42
```



POST crea recursos y devuelve información de dónde acceder a ellos.



Actualizar recursos: PUT vs PATCH

PUT (reemplazo completo)

```
PUT /assets/42
{
  "latitude": 41.0,
  "longitude": -3.5,
  "baseValue": 120000
}
```

```
csharp

[HttpPut("{id}")]
public IActionResult Update(int id, UpdateAssetDto dto)
{
    service.Update(id, dto);
    return NoContent();
}
```

Respuesta:

204 No Content



PUT asume que el cliente controla todo el estado del recurso.



Eliminar recursos: DELETE bien diseñado

Petición:

```
DELETE /assets/42
```

Acción:

```
csharp

[HttpDelete("{id}")]
public IActionResult Delete(int id)
{
    service.Delete(id);
    return NoContent();
}
```

Respuesta:

```
204 No Content
```

Buenas prácticas:

- no devolver cuerpo
- ser idempotente
- manejar el caso “no existe”

 *DELETE elimina, no ejecuta lógica adicional oculta.*



Errores comunes en controllers (muy reales)

Errores habituales:

- lógica de negocio dentro del controller
- acceso directo a la base de datos
- devolver siempre 200 OK
- endpoints con demasiadas responsabilidades

Mal ejemplo:

// controller de 500 líneas

Buen síntoma:

- controllers pequeños
- acciones cortas
- lógica delegada



Si el controller crece, el diseño necesita revisión.

7. Modelos, DTOs y Contratos de API

- Modelo interno vs contrato de API
- DTOs: definir exactamente qué entra y qué sale
- Flujo real: del JSON al dominio
- Controlar la salida: nunca devolver lo que no quieres
- Versionar contratos sin romper clientes

Modelo interno vs contrato de API (problema real)

Situación típica:

Tienes una entidad interna en el backend:

```
csharp

public class Asset
{
    public int Id { get; set; }
    public double Latitude { get; set; }
    public double Longitude { get; set; }
    public decimal BaseValue { get; set; }
    public DateTime CreatedAt { get; set; }
    public bool IsDeleted { get; set; }
}
```

 **Error común:** devolver esto directamente por la API.

Problemas:

- expones campos internos
- acoplas la API al dominio
- cualquier cambio rompe clientes

 *El modelo interno NO es el contrato externo.*

DTOs: definir exactamente qué entra y qué sale

Un **DTO (Data Transfer Object)** define el **contrato explícito** de la API.

Ejemplo de salida:

```
csharp

public class AssetResponseDto
{
    public int Id { get; set; }
    public decimal BaseValue { get; set; }
}
```

Ejemplo de entrada:

```
csharp

public class CreateAssetDto
{
    public double Latitude { get; set; }
    public double Longitude { get; set; }
    public decimal BaseValue { get; set; }
}
```

Ventajas:

- control total del contrato
- ocultas detalles internos
- puedes evolucionar el dominio sin romper la API

 *Un DTO es una promesa al consumidor de la API.*

Flujo real: del JSON al dominio

Petición HTTP real:

```
POST /assets
{
  "latitude": 40.4,
  "longitude": -3.7,
  "baseValue": 100000
}
```

Flujo en la API:

JSON



CreateAssetDto



Servicio de aplicación



Entidad de dominio

Ejemplo:

```
csharp
var asset = new Asset(
    dto.Latitude,
    dto.Longitude,
    dto.BaseValue
);
```



El controller traduce contratos, no construye dominio complejo.



Controlar la salida: nunca devolver lo que no quieres

Respuesta bien diseñada:

```
json
{
  "id": 42,
  "baseValue": 100000
}
```

Respuesta mal diseñada:

```
json
{
  "id": 42,
  "latitude": 40.4,
  "longitude": -3.7,
  "baseValue": 100000,
  "isDeleted": false,
  "createdAt": "2024-01-01"
}
```

Claves:

- devuelve solo lo necesario
- evita filtrar información interna
- piensa en quién consume la API



La API no es un espejo de la base de datos.

Versionar contratos sin romper clientes

Situación real:

- hoy el contrato es simple
- mañana necesitas añadir campos
- hay clientes antiguos en producción

Estrategias habituales:

- añadir campos opcionales
- crear nuevos DTOs
- versionar endpoints si es necesario

Ejemplo:

- /api/v1/assets
- /api/v2/assets

 *Cambiar contratos es caro; diseñarlos bien desde el inicio ahorra problemas.*

8. Códigos de estado HTTP y manejo de errores

- Códigos de estado HTTP: por qué importan de verdad
- Respuestas correctas en operaciones comunes
- Devolver errores de forma clara y consistente
- Validación automática con [ApiController]
- Qué NO debe hacer una API cuando ocurre un error

Códigos de estado HTTP: por qué importan de verdad

En una API REST, **el código HTTP es parte del contrato**, no un detalle técnico.

Un cliente interpreta primero:

- el **código de estado**
- luego el **cuerpo de la respuesta**

Ejemplo:

- 200 OK → todo correcto
- 404 NotFound → el recurso no existe

Error común:

- devolver siempre 200 OK y explicar el error en el body

 *Si el código es incorrecto, el cliente ya está confundido.*



Respuestas correctas en operaciones comunes

Obtener un recurso

GET /assets/42

- 200 OK → existe
- 404 Not Found → no existe

Crear un recurso

POST /assets

- 201 Created → creado correctamente
- 400 Bad Request → datos inválidos

Actualizar / eliminar

- 204 No Content → operación correcta sin cuerpo



Cada operación tiene un conjunto limitado de respuestas válidas.



Devolver errores de forma clara y consistente

Ejemplo de error mal diseñado:

```
json
{
  "error": "something went wrong"
}
```

Ejemplo mejor:

```
json
{
  "message": "Asset not found",
  "code": "ASSET_NOT_FOUND"
}
```

Buenas prácticas:

- mensajes claros
- estructura consistente
- no filtrar información interna



Los errores también son una API.

Validación automática con [ApiController]

ASP.NET Core puede validar automáticamente los inputs.

Ejemplo:

```
csharp

public class CreateAssetDto
{
    [Required]
    public decimal BaseValue { get; set; }
}
```

Si el cliente envía:

json

```
{ }
```

Respuesta automática:

400 Bad Request

Sin escribir código adicional en el controller.

 *Deja que el framework haga el trabajo repetitivo.*

Qué NO debe hacer una API cuando ocurre un error

Errores graves:

- devolver stack traces
- exponer nombres de tablas o columnas
- filtrar detalles internos del sistema

Ejemplo peligroso:

```
json
{
  "exception": "SQLException at line 42..."
}
```

 *La API protege al sistema, incluso cuando falla.*

9. Arquitectura interna de la API

- Qué hay detrás de un Controller
- Servicio de aplicación: dónde vive la lógica
- Repositorios: acceso a datos desacoplado
- Errores típicos de arquitectura en APIs ASP.NET Core
- Qué significa una API mantenible en el tiempo

Qué hay detrás de un Controller (visión real del backend)

Cuando un controller recibe una petición, **no debería resolverla solo**.

Estructura típica detrás:

Controller



Servicio de aplicación



Repositorio / infraestructura

Ejemplo mental:

- el controller recibe GET /assets/42
- delega la operación
- solo transforma el resultado en HTTP

 *El controller es la puerta de entrada, no el cerebro del sistema.*

Servicio de aplicación: dónde vive la lógica

Un **servicio de aplicación**:

- implementa casos de uso concretos
- coordina reglas de negocio
- decide qué hacer con los datos

Ejemplo conceptual:

```
csharp

public AssetDto GetById(int id)
{
    var asset = repository.Get(id);
    if (asset == null) return null;
    return mapper.Map(asset);
}
```

Claves:

- no conoce HTTP
- no devuelve IActionResult
- trabaja con conceptos del dominio

 *Aquí es donde el sistema “piensa”.*

Repositorios: acceso a datos desacoplado

Un **repositorio** abstrae el acceso a datos.

Responsabilidades:

- obtener entidades
- guardar cambios
- ocultar detalles de persistencia

Ejemplo:

- `public Asset Get(int id);`
- `public void Add(Asset asset);`

Ventajas:

- el resto del sistema no sabe si hay SQL, memoria o mocks
- facilita cambios futuros
- clarifica responsabilidades

 *El dominio no debe saber cómo se guardan las cosas.*

Errores típicos de arquitectura en APIs ASP.NET Core

Errores muy comunes:

- controllers accediendo a la base de datos
- lógica de negocio repartida en varios sitios
- DTOs usados como entidades
- dependencias circulares

Síntoma claro:

- “no sé dónde tocar para cambiar esto”

 *Cuando la arquitectura falla, el código se vuelve frágil.*

Qué significa una API mantenible en el tiempo

Una API mantenible:

- permite añadir endpoints sin romper otros
- permite cambiar la base de datos sin reescribir todo
- tiene responsabilidades claras por capa

Señales positivas:

- controllers pequeños
- servicios con nombres claros
- contratos estables

 *La mantenibilidad no se nota el primer día, se nota al sexto mes.*